

Fondements informatiques I

Cours 1: Variables et types

Sorina Ionica `sorina.ionica@uvsq.fr`

Bilal FAYE `bilal.faye@uvsq.fr`

Qu'est-ce qu'un algorithme ?

Un algorithme

Un algorithme est une méthode composée d'une suite finie d'étapes précises et ordonnées permettant de résoudre un problème.

Pour décrire un algorithme, on précise :

- Entrées (Input) : les données dont on dispose au départ
- Traitement : les étapes à effectuer, dans un ordre précis
- Sortie (Output) : le résultat obtenu

Exemple : calculer la moyenne de 3 notes

Input : 12, 15 et 17

Output : la moyenne des trois notes

Additionner les trois notes : $12 + 15 + 17 = 44$

Diviser la somme par 3 : $44/3 = 14,67$

Renvoyer 14,67

De l'algorithme au programme

Un **programme** est un ensemble d'instructions que l'ordinateur peut exécuter.

Pour écrire un programme, on utilise un **langage de programmation**.

L'algorithme précédent en Python

```
# Input : Les données en entrée
note1 = 12
note2 = 15
note3 = 17

# Traitement : Additionner les trois notes
somme = note1 + note2 + note3

# Traitement : Diviser la somme par 3
moyenne = somme / 3

# Output : Afficher le résultat
print(moyenne)
```

→ Un langage de programmation permet de traduire un algorithme en instructions exécutables par l'ordinateur.

Pourquoi programmer ?

Un algorithme devient utile lorsqu'il est exécuté par une machine.

Le programme traduit l'algorithme dans un **langage de programmation** que l'ordinateur peut comprendre et exécuter.

Exemples de programmes

- Application web
- GPS et navigation
- Jeux vidéo
- Applications mobiles
- Réseaux sociaux
- Intelligence artificielle

Langages de programmation

- Python
- Java
- C / C++
- JavaScript
- C#
- Kotlin / Swift

À retenir

Problème → **Algorithme** → **Programme** → **Exécution par l'ordinateur**

Dans cette UE

Nous utiliserons **Python** pour écrire nos programmes.

Quelques repères

- Créé par Guido van Rossum
- Première version en 1991
- Langage multiplateforme

Quelques caractéristiques

- Gratuit et open-source
- Windows, Linux, macOS, Android...
- Langage interprété

Langage interprété

Les instructions sont exécutées par un *interpréteur*, au fur et à mesure de l'exécution du programme.

Les variables

Qu'est-ce qu'une variable ?

Une **variable** est un nom associé à une valeur dans un programme.

Exemple en Python

```
x = 2           # x prend la valeur 2
y = x           # y prend la valeur de x
print(x)        # affiche 2
print(y)        # affiche 2
y = 3           # y prend maintenant la valeur 3
print(x, y)     # affiche 2 3
```

- Ici, x et y sont des variables de type entier.
- Modifier y ne modifie pas x.
- *Règle de style* : on met un espace après une virgule, mais pas avant.

Échanger deux variables

Objectif

Échanger les valeurs de deux variables sans perdre aucune valeur.

Première méthode : incorrecte

```
x = 2
y = 3
x = y      # x prend la valeur de y
y = x      # y prend la valeur de x
print(x, y) # affiche 3 3
```

Problème : la valeur initiale de x est perdue.

Échanger deux variables

Objectif

Échanger les valeurs de deux variables sans perdre aucune valeur.

Deuxième méthode : avec une variable temporaire

```
x = 2
y = 3
z = x      # sauvegarder la valeur de x
x = y      # x prend la valeur de y
y = z      # y récupère l'ancienne valeur de x
print(x, y) # affiche 3 2
```

Idée : utiliser z pour conserver temporairement une valeur.

Échanger deux variables

Objectif

Échanger les valeurs de deux variables sans perdre aucune valeur.

Troisième méthode : avec un tuple

```
x = 2
y = 3
x, y = y, x    # échange des deux valeurs
print(x, y)    # affiche 3 2
```

En Python : plusieurs valeurs peuvent être affectées simultanément.

Règles de nommage des variables

Comment choisir un nom de variable ?

Un nom de variable doit être **valide**, **lisible** et **explicite**.

Règles à respecter

- Lettres : a-z, A-Z
- Chiffres : 0-9
- Souligné : _
- Pas d'espace ni de caractère accentué
- Le nom ne commence pas par un chiffre

Attention

- Python est sensible à la casse
- toto $\neq$ toTo
- Ne pas utiliser de mot réservé
- Éviter les noms trop courts ou peu explicites

Exemples

✓ **Valide** : age nom_etudiant note1

✗ **Non valide** : 1note nom etudiant note-élève

✗ **À éviter** : print – mot déjà utilisé par Python

Les types de données

Qu'est-ce qu'un type de donnée ?

Le **type** d'une donnée détermine :

- les valeurs qu'elle peut prendre ;
- les opérations que l'on peut lui appliquer.

Les principaux types que nous allons manipuler au début

```
a = 5           # entier (int)
b = 3.14        # nombre flottant (float)
c = True        # booleen (bool)
d = "Bonjour"   # chaine de caracteres (str)
```

En Python

Le typage est **dynamique et implicite** :

- le type est déterminé automatiquement lors de l'affectation ;
- une même variable peut changer de type au cours du programme.

Les types de données

Connaître le type d'une donnée

Pour connaître le type d'une donnée, on peut utiliser la fonction `type()`.

Exemple en Python

```
a = 5
b = 3.14
c = True
d = "Bonjour"
print(type(a))           # <class 'int'>
print(type(b))           # <class 'float'>
print(type(c))           # <class 'bool'>
print(type(d))           # <class 'str'>
```

Les nombres

En Python, on manipule principalement deux types de nombres :

Entiers `int`

Nombres sans partie décimale.

Précision arbitraire : Python peut représenter des entiers très grands.

Flottants `float`

Nombres avec une partie décimale.

Le **point** est utilisé comme séparateur décimal.

Exemples

```
x = 12           # int
y = 3.14         # float
z = 2e+20        # float : 2 x 1020
```

Notation scientifique : 2e+20 signifie 2×10^{20} .

Opérateurs arithmétiques

Sur les nombres, Python permet d'utiliser les opérateurs arithmétiques usuels :

Opérateur	Opération	Exemple
+	Addition	5 + 2
-	Soustraction	5 - 2
*	Multiplication	5 * 2
/	Division	5 / 2
**	Puissance	5 ** 2
%	Reste	5 % 2

Exemple

```
a = 5 + 2      # 7
b = 5 * 2      # 10
c = 5 ** 2     # 25
d = 5 % 2      # 1
```

Division et reste

Trois opérateurs à distinguer

- / : division flottante
- // : division entière
- % : reste de la division

Exemple

```
print(11 / 2)      # 5.5  
print(11 // 2)     # 5  
print(11 % 2)      # 1
```

$$11 = 2 \times 5 + 1$$

Quotient : 5 Reste : 1

Résultats des opérations numériques

Le type du résultat dépend des types utilisés dans l'expression.

Entiers et flottants

```
print(type(5 + 2))      # <class 'int'>
print(type(5 + 2.0))    # <class 'float'>
print(type(7 * 1.0))    # <class 'float'>
print(type(10 ** 2))    # <class 'int'>
```

À retenir

La présence d'un float dans une opération numérique peut conduire à un résultat de type float.

Opérateurs d'affectation augmentés

Les opérateurs d'affectation augmentés permettent de modifier une variable à partir de sa valeur actuelle.

Ajouter une valeur

```
a = 2
b = 3
a += b      # équivaut à : a = a + b
print(a)    # 5
```

Incrémenter une variable

```
a += 1      # équivaut à : a = a + 1
print(a)    # 6
```

`-=` `*=` `/=` `//=` `%=`

Conversion de types

Une donnée peut être convertie dans un autre type lorsque cela est possible et pertinent.

Deux fonctions de conversion

- `int()` : conversion en entier
- `float()` : conversion en flottant

Exemple

```
x = 3
y = 3.14
a = float(x)           # 3 devient 3.0
b = int(y)             # 3.14 devient 3
print(type(a))         # <class 'float'>
print(type(b))         # <class 'int'>
```

Attention

`int()` ne fait pas un arrondi : la partie décimale est supprimée.

La précision des nombres flottants

Un nombre flottant est généralement représenté en machine par une approximation en **base 2**.

La précision étant limitée, certains nombres décimaux ne peuvent pas être représentés exactement.

Un résultat surprenant

```
print(0.1 + 0.2)
# 0.30000000000000004
print(0.1 + 0.2 == 0.3)
# False
```

À retenir

Un float est souvent une **approximation** de la valeur mathématique souhaitée.

Le type booléen

Une valeur booléenne

Une donnée de type `bool` ne peut prendre que deux valeurs :

True ou False

Les booléens permettent notamment de représenter le résultat d'une **condition**.

Exemple

```
a = 5 < 2
print(a)           # False
print(type(a))     # <class 'bool'>
```

Une comparaison produit une valeur booléenne.

Les opérateurs de comparaison

Les opérateurs de comparaison permettent de comparer deux valeurs. Le résultat est toujours un booléen : True ou False.

Opérateur	Signification	Exemple
>	supérieur à	5 > 2
<	inférieur à	5 < 2
>=	supérieur ou égal à	5 >= 5
<=	inférieur ou égal à	5 <= 2
==	égal à	5 == 5
!=	différent de	5 != 2

Exemple en Python

```
age = 20
print(age >= 18)      # True
print(age == 20)      # True
print(age != 20)      # False
```

Les opérateurs logiques

Les opérateurs logiques permettent de **combiner** ou **modifier** des valeurs booléennes.

not

Inverse une valeur.

```
not True = False
```

```
not False = True
```

and

True seulement si les deux valeurs sont vraies.

```
True and True = True
```

or

True si au moins une valeur est vraie.

```
True or False = True
```

Exemple

```
age = 20
majeur = age >= 18
print(majeur and True)      # True
print(majeur and False)     # False
print(not majeur)           # False
```

Construire une expression logique

Une expression logique peut combiner plusieurs comparaisons avec `and`, `or` et `not`.

Exemple

```
a = -0.5
p = (a <= 0)           # True
q = (a > 1)            # False
r = (a < 3)            # True
s = (a != 2)           # True
res = p or (q and r and s)
print(res)             # True
```

`p` est vrai, donc `p or (...)` est vrai.

À retenir

Une expression logique peut être construite à partir de **comparaisons** et d'**opérateurs logiques**.

Les chaînes de caractères

Qu'est-ce qu'une chaîne de caractères ?

Une **chaîne de caractères** (str) est une suite de caractères écrite entre guillemets simples ou doubles.

Exemples

```
s1 = "Bonjour"
s2 = 'Python'
s3 = "Bonjour Python"
print(s1)
print(type(s1))           # <class 'str'>
```

Simple ou double guillemets : les deux sont possibles.

Opérations sur les chaînes

Les chaînes de caractères peuvent être manipulées avec plusieurs opérations simples.

Exemple

```
s = "Bonjour"
print(s + " Python")    # concaténation
print(s * 3)            # répétition
print(len(s))           # longueur : 7
```

À retenir

- + assemble deux chaînes ;
- * répète une chaîne ;
- len() donne le nombre de caractères.

Accéder aux caractères d'une chaîne

Une chaîne est une suite de caractères. Chaque caractère possède une **position**, appelée indice.

Exemple

```
s = "Python"
print(s[0])      # P
print(s[1])      # y
print(s[5])      # n
```

P	y	t	h	o	n
0	1	2	3	4	5

Attention

En Python, le premier indice est **0**.

Quelques fonctions pour les chaînes

Une chaîne de caractères possède des **fonctions** permettant de la manipuler.

Exemple

```
s = "bonjour"
print(s.upper())      # "BONJOUR"
print(s.capitalize()) # "Bonjour"
print(s.replace("jour", "soir"))
```

Fonctions

Une fonction peut être associée à une donnée.

La notation `s.upper()` signifie : **appliquer la méthode** `upper()` à `s`.

Lire une donnée au clavier

La fonction `input()` permet de récupérer une donnée saisie par l'utilisateur.

Lire un texte

```
nom = input("Entrez votre nom : ")  
print("Bonjour", nom)
```

Attention

`input()` renvoie toujours une **chaîne de caractères** (`str`), même si l'utilisateur saisit un nombre.

Lire un entier

```
age = input("Entrez votre âge : ")  
age = int(age)           # conversion en entier  
print(age + 1)
```

À retenir : les types de données

Les types de base

int float bool str

En Python

- Le type est déterminé automatiquement.
- Une variable peut changer de type.
- `type()` permet de connaître un type.
- `int()`, `float()` et `str()` permettent de convertir une donnée.

Opérations essentielles

- Nombres : `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Booléens : `and`, `or`, `not`
- Chaînes : `+`, `*`, `len()`
- Comparaisons : `<`, `>`, `==`, `!=`, ...

Idée essentielle

Le type d'une donnée détermine les opérations que l'on peut effectuer sur celle-ci.